

gcp-erchat Repository Documentation (NLQ → Athena SQL API)

Repository: `gcp-erchat` **Service type:** FastAPI HTTP API **Primary purpose:** Convert a Natural Language Query (NLQ) into an AWS Athena (Presto/Trino) SQL `SELECT` statement using an LLM, validate it against security and permission constraints, optionally execute it in Athena, and return normalized JSON results.

This documentation focuses on:

- The API endpoints and their request/response payloads
- The permission model (UUID-pair → allowed Athena targets and tables)

1. High-level architecture and runtime flow

At runtime the `POST /nlq/execute` request goes through these phases:

1. **Request parsing / validation** (Pydantic) - Request is parsed into `NLQRequest`.
2. **Permission lookup** - Uses `context.account_uuid` and `context.property_uuid` to look up an allowlist entry. - If no entry exists: the request is rejected.
3. **Schema loading** (AWS Glue) - Loads table schemas for the chosen Athena target and compresses them into prompt text.
4. **Prompt building** - Creates an LLM prompt that includes strict SQL-only rules and the available schema.
5. **SQL generation (LLM)** - Runs Mistral 7B Instruct and extracts a `SELECT ...` statement.
6. **SQL validation and security checks** - Blocks destructive operations and non-allowed tables.
7. **Athena execution (optional)** - If not `dry_run`, execute the SQL in Athena and normalize results.
8. **Display hinting (heuristics)** - Suggests visualization type based on SQL and returned data.
9. **Logging** - Stores request and response payloads in a local JSON file (max 100 entries).

Endpoints are implemented in `app/main.py`.

2. Repository layout

Top-level folders:

- `app/`
 - Core FastAPI app, request/response models, permissions, SQL generation, validation, Athena execution, schema loading, and logging.
- `scripts/`
 - Convenience scripts (systemd service install, start/stop helpers).
- `archive/`
- Older/alternate endpoint examples.
 - `clis/`

- CLI snippets for curl/systemctl/git.

Key source files:

- ``app/main.py``
- FastAPI application and route handlers.
 - ``app/models.py``
- Pydantic request models.
 - ``app/permissions.py``
- Permission mapping and helpers.
 - ``app/security.py``
- SQL safety and table-allowlist validation.
 - ``app/prompt.py``
- Prompt template for SQL generation.
 - ``app/sqlcoder.py``
- Loads and runs the Mistral model; extracts SQL.
 - ``app/schema_loader.py``
- Reads Glue table schema; compresses it for the prompt.
 - ``app/athena_config.py``
- Athena target configuration (region/database/tables/S3 output/credentials).
 - ``app/athena_client.py``
- Executes queries via boto3 Athena; normalizes result rows.
 - ``app/request_logger.py``
- File-based request/response logging; ``logs`` endpoint reads from this.
 - ``app/display_hint.py``
- Heuristics for ``display.type``.

3. API: ``POST /nlq/execute``

3.1 Purpose

Convert natural language text into SQL (Athena dialect) and optionally execute it.

3.2 Request payload (`NLQRequest`)

The endpoint expects JSON matching `NLQRequest` in `app/models.py`.

3.2.1 Top-level fields

- `text`` **(string, required, min length 3)**
- The natural-language question.
- `context`` **(object, required)**
- Includes UUIDs required for access control.
- `sql`` **(object, required)**
- SQL configuration (dialect is currently locked to Athena).
- `execution`` **(object, required)**
- Controls whether Athena is called.
- `model`` **(object, required)**
- LLM generation parameters.
- `trace`` **(object, required)**

- Request tracing metadata.

3.2.2 `context`` object (`Context``)

- `account_uuid`` **(string, required)**
- Used for permission lookup.
- `property_uuid`` **(string, required)**
- Used for permission lookup.
- `user_role`` **(string, optional)**
- Present for future use; not currently enforced.
- `location_name`` **(string, optional)**
- Informational; not currently enforced.
- `language`` **(("en" | "zh" | "ms" | "ta", default: "en"))**

3.2.3 `sql`` object (`SQLConfig``)

- `dialect`` **(("athena", required)**
- Only Athena is supported. - Allowed tables are derived from permissions, not from the request.

3.2.4 `execution` object (`ExecutionConfig`)

- `dry\_run` *(boolean, default: true)*

- If `true`, the service generates and validates SQL but does not run Athena.

- `max\_rows` *(integer, default: 100)*

- Max rows to request from Athena.

- `timeout\_ms` *(integer, default: 5000)*

- Present in the request model but not currently enforced in Athena execution logic.

3.2.5 `model` object (`ModelConfig`)

- `name` *(string, default: "defog/sqlcoder-7b-2")*

- Present but not currently used by the runtime; the runtime loads `mistralai/Mistral-7B-Instruct-v0.3`.

- `temperature` *(number, default: 0.0)*

- Present but not used; generation is deterministic (`do\_sample=False`).

- `max\_tokens` *(integer, default: 512)*

- Used as `max\_new\_tokens` for model generation.

3.2.6 `trace` object (`Trace`)

- `request\_id` *(string, optional)*

- If omitted, the server generates one like `req-<uuid4>`.

- `source` *(string, default: "fcs1-ui")*

3.3 Request example

```
{
  "text": "how many incidents are there?",
  "context": {
    "account_uuid": "00000000-0000-0000-0000-000000000000",
    "property_uuid": "00000000-0000-0000-0000-000000000000",
    "language": "en"
  },
  "sql": {
    "dialect": "athena"
  },
  "execution": {
    "dry_run": false,
    "max_rows": 100,
    "timeout_ms": 5000
  },
  "model": {
    "max_tokens": 512
  },
  "trace": {
```

```

    "source": "manual-test"
  }
}

```

Notes:

- If you omit fields that have defaults inside nested objects, Pydantic will fill defaults — but the nested objects themselves are required.

4. API: `POST /nlq/execute` response payload

On success, the endpoint returns a JSON object with:

- `success` *(boolean)*
- `sql` *(object)*
- `execution` *(object)*
- `display` *(object)*
- `explanation` *(object)*
- `trace` *(object)*

4.1 Success response schema

4.1.1 `success`

- Always `true` for successful requests.

4.1.2 `sql` object

- `query` *(string)*

- The validated SQL query string (Athena dialect).

- `confidence` *(number)*

- Currently hard-coded to `0.90`.

4.1.3 `execution` object

- `executed` *(boolean)*

- `true` when Athena was executed (`dry\_run=false`), otherwise `false`.

- `row\_count` *(integer | null)*

- Count of returned rows when executed; `null` in dry-run mode.

- `data` *(object | null)*

- Normalized Athena result structure (see below), or `null` in dry-run mode.

Normalized Athena result (`execution.data`)

- ``columns` *(string[])*`
- ``rows` *(object[])*`

- Each row is a JSON object keyed by column name.

- ``row_count` *(integer)*`

Important behavior:

- Athena result normalization drops the header row and returns only data rows.
- If Athena returns 0 data rows (or only headers), the response becomes:

- ``columns: []`, `rows: []`, `row_count: 0``

4.1.4 ``display`` object

- ``type` *("table" | "line" | "bar" | "pie")*`

- Default is ``"table"``. - If executed and execution data exists, it uses heuristics in ``app/display_hint.py``.

4.1.5 ``explanation`` object Currently returned by the model runner as:

- ``summary` *(string)*`
- ``assumptions` *(string[])*`

4.1.6 ``trace`` object

- ``request_id` *(string)*`
- ``latency_ms` *(integer)*`

- LLM inference duration.

- ``athena_target` *(string)*`

- The target chosen from permissions.

- ``allowed_tables` *(string[])*`

- The allowlisted tables for this request.

4.2 Success response example

```
{
  "success": true,
  "sql": {
    "query": "SELECT COUNT(*) FROM incident_combine LIMIT 100",
    "confidence": 0.9
  },
  "execution": {
    "executed": true,
    "row_count": 1,
    "data": {
      "columns": ["_col0"],
      "rows": [{"_col0": "1523"}],
    }
  }
}
```

```

        "row_count": 1
    },
    "display": {
        "type": "table"
    },
    "explanation": {
        "summary": "SQL generated for Athena execution.",
        "assumptions": []
    },
    "trace": {
        "request_id": "req-a1b2c3d4-...",
        "latency_ms": 2345,
        "athena_target": "peninsula_incident",
        "allowed_tables": ["incident_combine"]
    }
}

```

5. API errors and failure modes

5.1 Permission denied

If the `(account\_uuid, property\_uuid)` pair is not found in permissions:

- HTTP status: `403`
- Body (FastAPI):

```

{
  "detail": "Access denied: No permissions for account <account_uuid> and property <property_uuid>"
}

```

5.2 Other failures (SQL generation/validation/execution)

The handler catches all exceptions and returns:

- HTTP status: `400`
- Body (FastAPI):

```

{
  "detail": "<error message>"
}

```

Examples that trigger `400`:

- LLM output does not contain a parseable `SELECT` query (`Empty SQL` after extraction).
- SQL contains forbidden keywords like `DROP`, `DELETE`, `UPDATE`, `INSERT`, `ALTER`, `TRUNCATE`.
- SQL contains unsupported Athena phrases like `DISTINCT ON` or `RETURNING`.
- SQL references a table not in the allowlist.
- Athena query fails (`FAILED` / `CANCELLED`).

Logging note:

- Even when the HTTP response is FastAPI's `{ "detail": ... }`, the service logs an internal `{ "success": false, "error": "..." }` structure for the failed request.

6. API: `GET /logs`

6.1 Purpose

Return the most recent request/response logs.

6.2 Query parameters

- `limit` *(integer, default 100)*

- The route clamps it to `min(limit, 100)`.

6.3 Response

```
{
  "total_logs": 42,
  "logs": [
    {
      "timestamp": "2025-12-22T12:34:56.789Z",
      "request_id": "req-...",
      "status_code": 200,
      "request": { "...": "..." },
      "response": { "...": "..." },
      "error": null
    }
  ]
}
```

6.4 Log storage and retention

- Stored in: `logs/api_requests.json` (relative to repo root)
- Retention: last 100 entries
- Concurrency safety: file operations are guarded by a thread lock

7. Permission model

Permissions are implemented as an in-memory allowlist in `app/permissions.py`.

7.1 How permissions are represented

A dictionary maps:

- Key: `(account_uuid: str, property_uuid: str)`
- Value:

- `athena_targets`: list of target names (strings) - `tables`: list of table names (strings)

Example (default “super user” entry):

```
(
```



```

"00000000-0000-0000-0000-000000000000",
"00000000-0000-0000-0000-000000000000"
): {
  "athena_targets": ["peninsula_incident", "londoner_granded"],
  "tables": ["incident_combine", "ldco_testing"]
}

```

7.2 How permissions are enforced in `/nlq/execute``

The `POST /nlq/execute`` handler:

1. Calls `get_allowed_access(account_uuid, property_uuid)``. 2. If missing → returns HTTP `403``. 3. Chooses the **first** Athena target from the permission entry: - `athena_target = access["athena_targets"][0]`` 4. Uses `access["tables"]`` as `allowed_tables``. 5. Validates the generated SQL against `allowed_tables`` (see Security section).

Important implication:

- Even if the permission entry contains multiple Athena targets, **the current implementation always uses the first target**.

7.3 Helper: `validate_access(...)``

`app/permissions.py`` also contains a helper `validate_access(account_uuid, property_uuid, requested_target, requested_tables)`` which can:

- Validate a requested target is in `athena_targets``.
- Validate each requested table is in `tables``.

However, the current API does not accept `requested_target`` or `requested_tables`` in the request payload, so this helper is not currently used by the live endpoint.

7.4 Production considerations (important)

- The permission store is currently hard-coded.
- UUIDs in the repo are example placeholders.
- Consider migrating permissions to a durable store (database/config service) and adding an admin update path if this service is used beyond prototype scope.

8. SQL security and validation

SQL checks are enforced by `app/security.py`` and by the Athena executor.

8.1 Validation rules (`validate_sql``)

`validate_sql(sql, allowed_tables, dialect)`` enforces:

- SQL must be non-empty.
- SQL must not contain destructive keywords:

- `DROP``, `DELETE``, `UPDATE``, `INSERT``, `ALTER``, `TRUNCATE`` (case-insensitive word boundaries)

- For dialect `athena``, blocks phrases:

- `DISTINCT ON`, `RETURNING`

- Extracts table names using regex: `from\s+<table>`

- Every extracted table must appear in `allowed\_tables`.

Limitations to be aware of:

- The table extraction is regex-based and only looks for `FROM <name>`.

- It may not detect tables in `JOIN` clauses. - It may not detect schema-qualified or quoted identifiers.

8.2 Athena execution restrictions

`app/athena\_client.py` enforces:

- Only queries whose string begins with `select` (case-insensitive) may be executed.

9. Display hinting (`display.type`)

When `dry\_run=false` and results exist, the API infers a recommended visualization type using heuristics in `app/display\_hint.py`.

Rules (simplified):

- If there are no rows → `"table"`.
- If the SQL and/or columns look time-like (date/time patterns), especially with `ORDER BY` → `"line"`.
- If exactly 2 columns, ≤ 10 rows, and SQL has aggregations (COUNT/SUM/AVG/MIN/MAX/STDDEV/VARIANCE) → `"pie"`.
- If SQL has `GROUP BY` and aggregations, and ≤ 50 rows → `"bar"`.
- Otherwise → `"table"`.

10. Athena targets and schema loading

10.1 Target configuration

Athena targets are defined in `app/athena\_config.py` as `ATHENA\_TARGETS`:

- `region`
- `database`
- `tables`
- `s3\_output`
- AWS credentials

The schema loader (`app/schema\_loader.py`) uses the `tables` list for each target to fetch schemas from AWS Glue.

10.2 Schema prompt compression

The Glue schema is converted into prompt-friendly text of the form:

```
- <table>: columns [col1 (type), col2 (type)]; partitions [partition1, partition2]
```

11. Model execution and SQL extraction

The model runner in ``app/sqlcoder.py``:

- Loads ``mistralai/Mistral-7B-Instruct-v0.3`` via HuggingFace Transformers.
- Generates a response and then extracts SQL:

- Strips markdown fences (``sql ...``) - Uses a regex to find a ``SELECT ...`` segment - Normalizes whitespace - Ensures a ``LIMIT 100`` exists (if missing, appends it)

Observed behavior:

- The service prints raw LLM output to stdout (useful for debugging, noisy in production).

12. Operational notes

12.1 Running locally

```
uvicorn app.main:app --host 0.0.0.0 --port 8080
```

12.2 Example curl

```
curl -s -X POST http://localhost:8080/nlq/execute \
-H 'Content-Type: application/json' \
-d '{
  "text": "how many incidents are there?",
  "context": {
    "account_uuid": "00000000-0000-0000-0000-000000000000",
    "property_uuid": "00000000-0000-0000-0000-000000000000",
    "language": "en"
  },
  "sql": {"dialect": "athena"},
  "execution": {"dry_run": true, "max_rows": 100, "timeout_ms": 5000},
  "model": {"max_tokens": 256},
  "trace": {"source": "curl"}
}'
```

13. Appendix: Endpoint-to-code map

- ``POST /nlq/execute`` → ``app/main.py`` (``execute``)

- Request model: ``app/models.py`` (``NLQRequest``) - Permissions: ``app/permissions.py`` (``get_allowed_access``) - Prompt: ``app/prompt.py`` (``build_prompt``) - LLM: ``app/sqlcoder.py`` (``run_sqlcoder``, ``extract_sql``) - SQL validation: ``app/security.py`` (``validate_sql``) - Athena execution: ``app/athena_client.py`` (``execute_query``) - Display hint: ``app/display_hint.py`` (``get_display_type``) - Logging: ``app/request_logger.py`` (``log_request``)

- ``GET /logs` → `app/main.py` (`view_logs`)`

- Logging store: ``app/request_logger.py` (`get_logs`, `get_log_count`)`